

Session Management Strategies

Java Backend Interview Reference

What Is a Session?

HTTP is stateless — each request is independent. A session maintains state between requests, tracking that multiple HTTP requests come from the same user. Session management determines how this state is stored, transmitted, and secured.

Strategy 1: Server-Side Sessions (Stateful)

How It Works

Server creates a session object on login, stores it (memory/Redis/DB), and sends the session ID to the client via a cookie. On each request, client sends cookie, server looks up session data.

```
// Spring Session + Redis (distributed sessions) @SpringBootApplication
@EnableRedisHttpSession(maxInactiveIntervalInSeconds = 1800) // 30 min public class App { ... } //
application.yml spring: session: store-type: redis data: redis: host: redis-host port: 6379 // Set-Cookie:
SESSION=abc123xyz; HttpOnly; Secure; SameSite=Lax // Redis key: spring:session:sessions:abc123xyz // ->
{userId, roles, cart, lastAccess, ...}
```

Attribute	Detail
Session ID	Cryptographically random, 128-bit token. Never predictable.
Storage options	In-memory (single server only), Redis (distributed, recommended), RDBMS (durable but slow), Hazelcast
Session fixation fix	Regenerate session ID on login: <code>request.changeSessionId()</code> . Prevents attacker from setting a known session ID.
Concurrent sessions	Limit to 1 active session per user: <code>ConcurrentSessionControlAuthenticationStrategy</code> in Spring Security
Absolute timeout	Session expires N minutes after creation regardless of activity. Prevents infinite sessions.
Idle timeout	Session expires N minutes after last activity. Slide on each request.
Pros	Easy revocation: delete session from Redis. Rich session data. Works with browsers. CSRF protection with SameSite cookie.
Cons	Stateful: server must store sessions. Scaling requires distributed store (Redis). Session store is a single point of failure if not HA.

Strategy 2: Token-Based Sessions (Stateless JWT)

How It Works

On login, server issues a signed JWT access token (15 min) and an opaque refresh token (7-30 days). Client stores refresh token in HttpOnly cookie, access token in memory. No server-side session storage.

```
// Login response: set refresh token as HttpOnly cookie + return access token in body
@PostMapping("/auth/login") public ResponseEntity<LoginResponse> login(@RequestBody LoginRequest req,
HttpServletResponse res) { AuthResult auth = authService.authenticate(req); // Refresh token: HttpOnly,
Secure, SameSite=Strict cookie Cookie refresh = new Cookie("refreshToken", auth.refreshToken());
refresh.setHttpOnly(true); refresh.setSecure(true); refresh.setPath("/api/auth/refresh"); refresh.setMaxAge(7
* 24 * 3600); // 7 days res.addCookie(refresh); // Access token: return in body (SPA stores in memory) return
ResponseEntity.ok(new LoginResponse(auth.accessToken())); } // Refresh endpoint @PostMapping("/auth/refresh")
public ResponseEntity<TokenResponse> refresh( @CookieValue("refreshToken") String refreshToken) { String
newAccess = authService.refreshAccessToken(refreshToken); return ResponseEntity.ok(new
TokenResponse(newAccess)); }
```

Component	Storage	TTL	Why
Access token (JWT)	JS memory (SPA) or secure cookie	15 min	Short life limits damage if stolen
Refresh token	HttpOnly cookie only	7-30 days	HttpOnly prevents XSS; Secure prevents HTTP sniff
Access token (mobile)	Secure enclave (iOS Keychain, Android Keystore)	15 min	OS-protected storage, not accessible to other apps

Strategy 3: OAuth2 Token Sessions

When using an external IdP (Keycloak, Auth0, Okta, Google), the session is managed by the IdP. Your API acts as a Resource Server that validates tokens.

```
// Spring Boot as OAuth2 Resource Server @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { return http .oauth2ResourceServer(o -> o.jwt(j -> j . jwkSetUri("https://idp.example.com/.well-known/jwks.json") ) ) .sessionManagement(s -> s.sessionCreationPolicy(STATELESS)) .build(); } // IdP serves JWKS (public keys). Your API validates JWT signature. // No DB call needed. Token revocation: check token introspection endpoint or maintain local blocklist.
```

Strategy Comparison

Criteria	Server Session	JWT Stateless	OAuth2 IdP
State storage	Server (Redis)	Client (token)	IdP + client
Revocation	Instant — delete from Redis	Requires blocklist or wait for expiry	IdP revocation + local blocklist
Scalability	Requires distributed session store	Fully stateless — infinite scale	Stateless — scales well
CSRF risk	High (mitigate with SameSite cookie)	Low (Bearer header not auto-sent)	Low
XSS risk	Low (HttpOnly session cookie)	High if in localStorage	Moderate
SSO support	Complex — requires session sharing	Partial — pass JWT	Native — IdP handles SSO
Best for	Traditional web, server-rendered apps	REST APIs, microservices, mobile	Multi-app SSO, enterprise, B2C

Session Security Hardening

Threat	Attack	Defense
Session Hijacking	Attacker steals session ID via XSS or network sniff, impersonates user	HttpOnly + Secure + SameSite cookie HTTPS only Bind session to IP/User-Agent (optional, breaks proxies)
Session Fixation	Attacker sets a known session ID before login; user logs in with that ID; attacker reuses it	Regenerate session ID on login: request.changeSessionId() Never accept session IDs from URL parameters
CSRF	Malicious site submits a forged request using victim's session cookie (sent automatically)	SameSite=Strict cookie CSRF token (Double Submit Cookie) Verify Origin header Don't use session cookies on REST APIs — use Bearer token
Session Prediction	Attacker guesses session ID if it's not random enough	Use cryptographically secure random 128-bit session ID Java: new SecureRandom(); never Math.random()
Expired session reuse	User's old session used after logout	Invalidate server-side on logout: sessionRegistry.expireNow() Client deletes cookie on logout
Concurrent sessions	User logged in on multiple devices unknowingly (after credential theft)	Limit to 1 active session, or show active sessions to user for self-service revocation

Cookie Attributes Explained

Attribute	Value	Effect
HttpOnly	(flag)	Cookie not accessible via document.cookie — prevents XSS theft
Secure	(flag)	Cookie only sent over HTTPS — prevents network sniffing

Attribute	Value	Effect
SameSite	Strict / Lax / None	Strict: never sent cross-site. Lax: sent on top-level navigation GET. None: always sent (requires Secure). Controls CSRF.
Path	/api/auth	Cookie only sent to matching path prefix — limits exposure
Domain	.example.com	Cookie sent to example.com and all subdomains
Max-Age	Seconds	Cookie lifetime. 0 = delete immediately. Absent = session cookie (deleted on browser close).
Expires	Date string	Absolute expiry. Max-Age takes precedence if both set.

Spring Security Session Configuration

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { return http // For traditional web app with server sessions: .sessionManagement(s -> s .
sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED) .sessionFixation(sf -> sf.changeSessionId()) // regenerate on login .maximumSessions(1) // one session per user .maxSessionsPreventsLogin(false) // new login expires old session ) .rememberMe(r -> r .tokenRepository(persistentTokenRepository()) // DB-backed remember-me .tokenValiditySeconds(14 * 24 * 3600) // 14 days .key("remember-me-secret") ) .logout(l -> l .logoutUrl("/api/auth/logout") .deleteCookies("SESSION", "JSESSIONID") .invalidateHttpSession(true) .logoutSuccessHandler((req, res, auth) -> res.setStatus(200)) ) .build(); }
```

Distributed Session with Spring Session + Redis

```
// Multiple app instances share sessions via Redis // Session lifecycle: // 1. POST /login ->
SessionRepository.save(session) -> Redis // 2. GET /api/me -> SessionRepository.findById(sessionId) -> Redis lookup // 3. POST /logout -> SessionRepository.deleteById(sessionId) -> Redis delete // Manual session operations @Autowired FindByNameSessionRepository<? extends Session> sessionRepo; // Find all sessions for a user (requires SPRING_SECURITY_CONTEXT index) Map<String, ? extends Session> sessions = sessionRepo.findByPrincipalName(username); // Expire all sessions for user (force logout everywhere) sessions.forEach((id, s) -> sessionRepo.deleteById(id));
```

Interview Q&A;

Q1: What is the difference between a session and a JWT?

A session stores user state on the server (Redis/DB); the client only holds an opaque session ID (cookie). JWT is self-contained — all user state is in the token, stored on the client. Sessions are easy to revoke (delete from Redis) but require distributed state. JWTs are stateless and scalable but hard to revoke before expiry.

Q2: What is session fixation and how do you prevent it?

An attacker visits your login page, gets a session ID, tricks the victim into using that URL with the session ID, victim logs in — now attacker and victim share the same session ID. Prevention: on successful login, invalidate the old session and issue a new session ID (Spring: sessionFixation().changeSessionId()). Never accept session IDs from URL parameters.

Q3: What is CSRF and how does SameSite cookie attribute prevent it?

CSRF tricks the victim's browser into sending an authenticated request to your API — cookies are sent automatically. SameSite=Strict prevents cookies from being sent on any cross-site request. SameSite=Lax only allows cookies on top-level navigations (not AJAX). Either prevents malicious third-party sites from sending authenticated requests. For REST APIs, using Bearer tokens in Authorization header (not cookies) is inherently CSRF-safe because headers aren't auto-sent by browsers.

Q4: Why store access tokens in memory (not localStorage) for SPAs?

localStorage is accessible via JavaScript — any XSS vulnerability in the page can steal tokens from localStorage. Memory storage (JS variable) is lost on page refresh but is not accessible to injected scripts from other origins. Use a short-lived access token in memory + HttpOnly refresh token cookie. On page refresh, the SPA calls /auth/refresh, which reads the HttpOnly cookie (not accessible to JS) and returns a new access token.

Q5: How do you implement "remember me" securely?

Do NOT persist the session indefinitely. Instead: (1) Issue a long-lived remember-me token (random + hashed); (2) Store hashed token in DB with userId and expiry; (3) Send as separate HttpOnly cookie; (4) On each use, rotate the token (new random, update DB) — this detects token theft (if old token is used after rotation, revoke all sessions for user). Spring Security has PersistentTokenRepository for this.

Q6: How do you handle session management across multiple app instances?

Use Spring Session with Redis as the session store. All instances share the same Redis — any instance can look up any session. The session ID in the cookie routes to Redis, not to a specific instance. Configure Redis Sentinel or Cluster for HA. Without this, sticky sessions (routing same user to same instance) work but create imbalance and fail during deployments.

Q7: How would you force logout of all sessions for a user?

With server-side sessions (Spring Session + Redis): call `sessionRepo.findByPrincipalName(username)` to get all sessions, then `sessionRepo.deleteByld(id)` for each. With JWT: maintain a per-user "sessions invalidated at" timestamp in Redis; on each token validation, check if `iat` (issued-at) is before that timestamp — if so, reject the token. This effectively revokes all JWTs issued before the logout event.

Q8: What session timeouts should you configure and why?

Configure both: (1) Idle timeout — session expires after N minutes of inactivity (e.g., 30 min). Protects unattended screens. (2) Absolute timeout — session expires N hours after creation regardless of activity (e.g., 8 hours). Limits session lifetime even for active users, reducing stolen-session risk. For sensitive apps (banking), shorter: 10 min idle, 4 hr absolute. Warn users before idle timeout with a JavaScript countdown.